

Trabajo Práctico Final

Implementación de Blog Personal sobre Infraestructura Virtualizada

Sergio Rodrigo Zurita

2026

Universidad Tecnológica Nacional
Facultad Regional Tucumán
Ingeniería en Sistemas de Información
Cátedra de Virtualización

Índice

1. Introducción	3
2. Objetivos del proyecto	3
3. Arquitectura general	3
3.1. Topología lógica	3
3.2. Direccionamiento utilizado	4
4. Stack tecnológico elegido	4
4.1. Por qué no se eligió Next.js como solución final	4
5. Modelo de datos	5
5.1. Datos iniciales	5
6. Implementación funcional del blog	5
6.1. Sitio público	5
6.2. Panel privado de administración	6
7. Seguridad aplicada	6
8. Flujo de desarrollo y despliegue	6
8.1. Entorno local	6
8.2. Despliegue en Proxmox	7

9. Problemas encontrados y resolución	7
9.1. Problema 1: la app funcionaba por IP, pero no bien por NAP	7
9.2. Problema 2: NAP devolvía 501 para CSS y JS	7
9.2.1. Solución aplicada	8
9.3. Problema 3: imágenes locales sin cargar	8
9.3.1. Solución aplicada	8
9.4. Problema 4: diferencias entre entorno local y entorno institucional	8
10. Conclusión	9

1. Introducción

El presente informe describe el diseño, desarrollo, despliegue y ajuste de un servicio de blog personal implementado como trabajo práctico final de la cátedra de Virtualización. El proyecto fue pensado para funcionar en una infraestructura basada en contenedores LXC sobre Proxmox, respetando una separación clara entre servidor de aplicaciones y servidor de base de datos.

El objetivo no fue solamente construir una página web, sino resolver un caso real de publicación de una aplicación detrás de una infraestructura institucional con restricciones de proxy reverso, subredes públicas y recursos limitados. Durante el desarrollo se tomaron decisiones técnicas orientadas a priorizar simplicidad operativa, bajo consumo de recursos, despliegue predecible y compatibilidad con el entorno provisto por la facultad.

2. Objetivos del proyecto

Los objetivos principales del trabajo fueron los siguientes:

- Implementar un blog personal funcional con contenido dinámico.
- Separar el servidor de aplicaciones del servidor de base de datos en contenedores independientes.
- Desplegar la solución sobre Proxmox utilizando dos contenedores LXC.
- Administrar publicaciones y datos personales desde un panel privado.
- Aplicar buenas prácticas mínimas de seguridad: contraseñas hashadas, consultas preparadas, protección CSRF y escape de salida.
- Resolver las limitaciones impuestas por el NAP institucional para publicación bajo subred.

3. Arquitectura general

La arquitectura final del sistema quedó compuesta por dos contenedores:

Contenedor	Tecnología principal	Responsabilidad
Aplicación	Apache + PHP + PDO	Servir el blog público, el login, el dashboard y los handlers de escritura
Base de datos	MariaDB	Persistir publicaciones, usuario administrador y datos de perfil

3.1. Topología lógica

Internet / NAP institucional
v

```
nap.frt.utn.edu.ar/<identificador>
    |
    v
Contenedor APP (Apache + PHP)
    |
    v
Contenedor DB (MariaDB)
```

3.2. Direccionamiento utilizado

Durante las pruebas y el despliegue se trabajó con una estructura de red del tipo:

- Contenedor APP: 172.16.90.157
- Contenedor DB: 172.16.90.158
- Publicación institucional: `nap.frt.utn.edu.ar/<identificador>`

La aplicación funcionó correctamente por IP directa dentro de la red interna del entorno Proxmox. El desafío adicional apareció al publicarla detrás del NAP, que imponía una subruta y restricciones particulares sobre assets estáticos.

4. Stack tecnológico elegido

La implementación final se construyó con:

- **Servidor web:** Apache 2
- **Lenguaje:** PHP 8.2
- **Acceso a datos:** PDO con prepared statements
- **Base de datos:** MariaDB
- **Frontend:** HTML, CSS y JavaScript con animaciones GSAP
- **Entorno local de desarrollo:** Docker Compose
- **Entorno final:** Proxmox con dos contenedores LXC

4.1. Por qué no se eligió Next.js como solución final

Durante el análisis se evaluó una alternativa basada en Next.js. Sin embargo, esa solución requería:

- Node.js instalado en el contenedor de aplicación.
- Build externo y posterior empaquetado en modo `standalone`.
- Scripts de despliegue adicionales para actualizar y reiniciar el runtime Node.
- Workarounds especiales para sortear bloqueos del proxy sobre archivos `.css` y `.js`.

Si bien era técnicamente viable, implicaba más complejidad operativa y más puntos de falla. Se decidió entonces una solución basada en Apache + PHP por ser más natural para el entorno, más simple de mantener y más alineada con un despliegue directo sobre servidor web clásico.

5. Modelo de datos

La base de datos quedó organizada alrededor de tres tablas principales:

Tabla	Descripción
publicaciones	Guarda las publicaciones del blog: título, contenido, categoría y fecha
usuarios	Guarda credenciales del administrador y el flag de cambio obligatorio de contraseña
perfil	Guarda la información editable del hero y la sección “Sobre mí”

5.1. Datos iniciales

Se preparó un `init.sql` con:

- estructura de tablas,
- publicaciones iniciales,
- datos iniciales del perfil.

El usuario administrador no se cargó por SQL plano, sino con un script PHP específico para poder usar `password_hash()`.

6. Implementación funcional del blog

La aplicación quedó dividida en dos partes bien diferenciadas.

6.1. Sitio público

El sitio público incluye:

- Hero principal con imagen de fondo y presentación.
- Sección “Sobre mí” con foto y descripción personal.
- Sección de publicaciones renderizada desde MariaDB.
- Links sociales y datos complementarios.

6.2. Panel privado de administración

El panel privado incluye:

- Login de administrador.
- Alta, edición y eliminación de publicaciones.
- Edición de datos del perfil.
- Cierre de sesión.

7. Seguridad aplicada

Aunque se trata de un proyecto académico, se aplicaron medidas básicas reales de seguridad:

- Contraseñas almacenadas con `password_hash()`.
- Verificación con `password_verify()`.
- Consultas preparadas con PDO para evitar inyección SQL.
- Tokens CSRF en formularios con operaciones de escritura.
- Escape de salida con `htmlspecialchars()` para mitigar XSS.
- Regeneración del identificador de sesión al autenticar.

Además, se implementó una mejora adicional: el usuario inicial `admin/admin123` queda forzado a cambiar su contraseña en el primer ingreso. Esto se resolvió agregando un flag `requiere_cambio_password` en la tabla `usuarios` y una pantalla dedicada para actualizar la clave antes de permitir acceso completo al dashboard.

8. Flujo de desarrollo y despliegue

8.1. Entorno local

Para desarrollo local se utilizó Docker Compose con dos servicios:

- `app`: contenedor PHP/Apache.
- `db`: contenedor MariaDB.

Esto permitió validar la lógica del blog, los seeds, el login y el dashboard antes de pasar al entorno Proxmox.

8.2. Despliegue en Proxmox

En Proxmox el procedimiento general fue:

1. Clonar el repositorio en el contenedor APP.
2. Crear el archivo `.env` con la IP del contenedor DB.
3. Crear la base y el usuario en MariaDB.
4. Cargar `db/init.sql`.
5. Ejecutar `db/init-admin.php` desde el contenedor APP.
6. Configurar Apache para exponer variables necesarias como `DB_HOST` y luego `APP_BASE_PATH`.

9. Problemas encontrados y resolución

Esta fue la parte más importante del trabajo, porque aparecieron problemas reales de infraestructura y no solo de código.

9.1. Problema 1: la app funcionaba por IP, pero no bien por NAP

La aplicación respondía correctamente en:

```
http://172.16.90.157
```

Sin embargo, públicamente se accedía mediante una subruta del tipo:

```
https://nap.frt.utn.edu.ar/38249507/
```

Esto obligó a adaptar la aplicación a un **base path** configurable. Para eso se implementaron helpers centrales del estilo:

- `app_base()`
- `app_url()`
- `app_redirect()`

Con esto fue posible generar links, formularios, assets y redirecciones respetando el prefijo público asignado por NAP.

9.2. Problema 2: NAP devolvía 501 para CSS y JS

Se comprobó experimentalmente que el NAP devolvía 501 Not Implemented para archivos como:

```
/38249507/assets/css/style.css  
/38249507/assets/js/main.js
```

Lo más importante fue que esos requests muchas veces **no llegaban** al Apache del contenedor, lo que permitió concluir que el bloqueo estaba ocurriendo a nivel proxy.

9.2.1. Solución aplicada

Se implementó un workaround seguro y pragmático:

- `assets/style.php` para servir el CSS.
- `assets/main.php` para servir el JavaScript.

En lugar de pedir archivos `.css` y `.js` directos, el navegador pasa a pedir endpoints PHP que devuelven esos contenidos con sus `Content-Type` correctos.

9.3. Problema 3: imágenes locales sin cargar

Una vez resuelto CSS y JS, todavía quedaban fallas en imágenes locales, especialmente:

- la imagen de fondo del hero,
- la imagen de perfil.

El motivo fue similar: las imágenes seguían apuntando a rutas físicas directas como `/assets/img/...` y el NAP no las trataba igual que al HTML principal.

9.3.1. Solución aplicada

Se crearon endpoints dedicados:

- `assets/hero-image.php`
- `assets/profile-image.php`

Desde ese momento, tanto el CSS como el HTML referencian imágenes a través de PHP y el sitio logró mantener consistencia en el acceso público detrás del NAP.

9.4. Problema 4: diferencias entre entorno local y entorno institucional

En local, con Docker Compose, los assets y rutas funcionaban de forma directa. En cambio, en el entorno institucional apareció una capa adicional de complejidad:

- publicación bajo subruta,
- posibles reglas del reverse proxy por extensión,
- redirecciones y sesiones condicionadas por el path público.

Esto mostró una diferencia importante entre **hacer funcionar una app** y **hacerla compatible con una plataforma real de publicación**.

10. Conclusión

El trabajo permitió construir un blog personal funcional con administración real de contenido, separación entre aplicación y base de datos, y despliegue sobre contenedores virtualizados en Proxmox.

Más allá del desarrollo del sitio, el aprendizaje principal estuvo en resolver problemas de infraestructura reales: publicación por subruta, compatibilidad con reverse proxy, diferencias entre acceso interno por IP y acceso público institucional, y necesidad de adaptar la generación de URLs y el servido de assets.

La solución final demostró que, con una arquitectura simple pero bien pensada, es posible construir una aplicación sólida para el contexto del TP sin depender de runtimes complejos. También dejó como hallazgo importante que parte de la complejidad no estuvo en el código del blog en sí, sino en entender y acomodar la aplicación al comportamiento del NAP institucional.